

1 概述

题型：选择题 + 大题

占比：60%

考察范围：除 BCNF 的分解不考

2 基本概念

2.1 导论

- Data;
- Data dictionary(数据字典): 数据的数据, 即**元数据(Metadata)**
- DBMS;
- 数据库系统: 一般由数据库、DBMS、应用系统、DBA 和用户构成。
- **Data Model:** A collection of conceptual tools for describing *data, data relationships, data semantics, and consistency constraints.*
 - 由**数据结构**、**数据操作**和**完整性约束**三部分组成;
- 三层模式结构:
 - **Physical level:** Describes how the data are actually stored , including complex low-level data structures in detail. (数据是如何存储的)
 - 对应**物理模式/内模式**, 使用物理模型, e.g. 顺序、B+树、哈希存储;
 - **Logical level:** Describes what data are stored in the database, and what relationships exist among those data.
 - 对应逻辑模式, 使用逻辑模型, e.g. 关系模型、面向对象模型;
 - **View level:** Describes only part of the entire database , existing to simplify the user's interaction with the system.
 - 对应外模式;
- 数据独立性:
 - **物理数据独立性:** 应用程序不依赖于物理模式, 物理模式的修改不影响前端应用;
 - **逻辑数据独立性:** 逻辑方式的改变, 前端应用**不一定**需要进行更改, 只需要更改视图层和逻辑层之间的**映射** (The View/Logical mapping) ;
- 数据库语言:

- DDL (Data Definition Language, 数据定义语言): 定义数据库模式, 关系属性等;
- DML (Data Manipulation Language, 数据操纵语言, also known as query language);
- **Instance(实例)**: The collection of information stored in the database *at a particular moment* is called an instance of the database. (数据库在某一时刻的一个映像);
- **Schema(模式)**: The overall design of the database is called the database schema. (数据库的总体设计)

2.2 关系模型

- **Relation(关系)**: In the relational model the term relation is used to refer to a *table*.
- **Relation instance(关系实例)**: We use the term relation instance to refer to a specific instance of a relation, that is, containing a specific set of rows.
- **Relation schema(关系模式)**: refers to its *logical design*, including its attributes, and optionally the types of the attributes and constraints on the relation such as primary and foreign-key constraints.
 - 即关系数据结构、关系数据操作和关系数据完整性约束;
- **Key(键/码)**:
 - **Super Key(超键)**: 能唯一确定关系中的一个元组的属性集合;
 - **Candidate Key(候选键)**: 在超键的基础上满足最小性;
 - **Primary Key(主键)**: 从侯选键中选择一个;
 - **Foreign Key(外键)**;
- **完整性约束**:
 - 实体完整性约束(Integrity Constraint): 主键不为 NULL;
 - 参照完整性约束(referential integrity constrain): 外键列的值, 要么为 NULL, 要么必须在被引用表的主键列中存在;
 - 用户自定义完整性约束: 非空约束 not null, 唯一性约束 unique, 自定义约束 check, 默认值等;
- **Relational Query Languages(关系查询语言)**:
 - **Functional Query Language (函数式查询语言)** :
 - 把查询看成“函数的组合与求值”, 查询被表达为一系列函数调用;
 - e.g. 关系代数 `$\pi$  name (  $\sigma$  dept='CS' (Student) )`;

- **Declarative Query Language (声明式查询语言) :**
 - 只描述“要什么”，不描述“怎么做”；
 - e.g. 关系演算 (Relational Calculus) , SQL (执行策略由查询优化器处理) ；
- **“纯”查询语言 (Pure Query Languages) :**
 - 基于数学形式定义的、与实现无关的查询语言，e.g.
 - **关系代数 (Relational Algebra)**
 - **元组关系演算 (Tuple Relational Calculus, TRC)**
 - **域关系演算 (Domain Relational Calculus, DRC)**
- **关系代数：**（内容转载自 YouXam）
 - 六大基本关系代数：选择、投影、并、差、笛卡尔积、重命名。
 - 其余关系代数均可由基本关系代数推导得出；

1. 选择 Select:

$\sigma_{\theta(R)}$ ：从关系 (R) 中选取满足特定条件（谓词）的元组（行）。

例如： $\sigma_{age>20}(\text{Student})$ 选取 Student 表中年龄大于 20 的元组。

2. 投影 Project:

$\Pi_{A_1, A_2, \dots, A_n}(R)$ ：从关系 (R) 中选取特定属性（列）。

例如： $\Pi_{name, age}(\text{Student})$ 选取 Student 表中的姓名和年龄。

3. 交、并、差:

- $R \cap S$ ：返回两个关系的交集。
- $R \cup S$ ：返回两个关系的并集。
- $R - S$ ：返回属于 R 但不属于 S 的元组。

4. 笛卡尔积:

$R \times S$: 返回两个关系的笛卡尔积。

例如: $\text{Student} \times \text{Dept}$ 返回 Student 表和 Dept 表的笛卡尔积。

5. 连接:

- 等值连接:

$R \bowtie_{\theta} S$: 根据连接条件 θ 连接两个关系。

例如: $\text{Student} \bowtie_{\text{Student.DeptID} = \text{Dept.DeptID}} \text{Dept}$ 返回 Student 表和 Dept 表根据 DeptID 连接的结果。

- 自然连接:

$R \bowtie S$: 根据两个关系的公共属性连接。

等值连接和自然连接称为内连接。自然连接是一种特殊的等值连接,它要求两个关系中进行比较的分量必须是相同的属性组,并且在结果中把重复的属性列去掉。

- 外连接:

- 左外连接: $R \ltimes S$ 返回 R 中的所有元组和 S 中匹配的元组,没有匹配的用 NULL 填充。

例如 $\text{Employees} \ltimes \text{Departments}$ 返回所有员工的信息和员工的部门信息,即使员工没有部门也会填充 NULL。

- 右外连接: $R \ltimes S$ 返回 S 中的所有元组和 R 中匹配的元组,没有匹配的用 NULL 填充。
- 全外连接: $R \ltimes S$ 返回 R 和 S 中的所有元组,没有匹配的用 NULL 填充。

6. 重命名:

$\rho_{\text{new_name}}(R)$: 重命名关系 R 。

例如: $\sigma_{\text{instructor.salary} < \text{d.salary}}(\text{instructor} \times \rho_d(\text{instructor}))^1$

7. 聚集:

`group_by` γ `expr as name` (*R*): 根据属性 `group_by` 分组, 并根据 `expr` 计算, 重命名为 `name`。

例如:

```
dept_id  $\gamma$  average(salary) as avg_salary (Student)
```

返回按照 `dept_id` 分组的平均工资。

8. 除法: 小概率涉及, $R \div S$ 表示, 找出那些在 *R* 中, 对 *S* 中“所有值”都成立的元组;

3 SQL 语句

3.1 DML 数据操纵语言

DML: A language that enables users to *access or manipulate data* as organized by the appropriate data model, which includes a query language and commands to *insert tuples into, delete tuples from, and modify tuples* in the database.

3.1.1 查询语句基本结构

```
1 SELECT [DISTINCT] 列名/表达式 [AS 别名], ...
2 FROM 表名 [AS 别名], ...
3 WHERE 条件
4 GROUP BY 列名
5 HAVING 条件
6 ORDER BY 列名 [ASC|DESC];
```

其中:

- select 子句注意:
 - 重命名: `A as A'`;
 - 默认保留重复, 去重使用 `select distinct`;
- where 子句注意:
 - and / or / not
 - between ... and ...
 - null 值处理:
 - `null  $\neq$  null` 在内的任何值;
 - 任何比较 + `null` = `unknown`;
 - `unknown` 视为 `false`;

3.1.2 聚集函数 (Avg / Sum / Count / Min / Max)

```
1 SELECT DeptID, COUNT(StudentID) AS NumOfStudents
2 FROM Student
3 GROUP BY DeptID
4 HAVING COUNT(StudentID) > 10;
```

注意:

- 除 `count(*)` 外: 忽略 **null**
- 如果全是 null:
 - count → 0
 - 其他 → null
- **having** 在分组后执行, **where** 在分组前执行, 具体可以看前面的基本结构, 按照 `where - groupby - having` 的顺序;
- **select** 中非聚集属性, 必须出现在 **group by** 中

错误示例:

```
1 select dept_name, ID, avg(salary)
2 from instructor
3 group by dept_name;
4 -- ID 没有出现在 group by 中
```

3.1.3 集合操作 (Union / Intersect / Except)

用于将查询结果进行合并、取交、作差运算:

```
1 -- 并操作
2 SELECT StudentID FROM CS_Student
3 UNION
4 SELECT StudentID FROM Math_Student;
5
6 -- 交操作
7 SELECT StudentID FROM CS_Student
8 INTERSECT
9 SELECT StudentID FROM Math_Student;
10
11 -- 差操作
12 SELECT StudentID FROM CS_Student
13 EXCEPT
14 SELECT StudentID FROM Math_Student;
```

3.1.4 子查询

以下是 where 子句中的子查询, from 子句子查询出现频率较低:

1. 集合成员资格 (in / not in)

```
1 where course_id in (select course_id ...)
```

2. 集合比较 (any / some / all)

```
1 SELECT S.StudentID
2 FROM Student S
3 WHERE S.Age > ANY (
4     SELECT Age FROM Student WHERE DeptID = 3
5 );
```

- any / some: 存在一个即可，至少一个；
- all: 全部；

3. 空关系测试 (exists / not exists)

```
1 where exists (select * ...)
```

如果存在至少一条满足子查询的记录，则对应记录被选出。

4. 重复元组存在性测试 (unique / not unique)

```
1 SELECT T.course_id
2 FROM course AS T
3 WHERE UNIQUE (
4     SELECT R.course_id
5     FROM section AS R
6     WHERE T.course_id = R.course_id AND R.year = 2017
7 );
```

5. 中间变量保存 (with) :

```
1 WITH DeptCount AS (
2     SELECT DeptID, COUNT(*) AS NumOfSt
3     FROM Student
4     GROUP BY DeptID
5 )
6 SELECT *
7 FROM DeptCount
8 WHERE NumOfSt > 10;
```

相当于新建了一个名为 `DeptCount (DeptID, NumOfSt)` 的关系用于后续使用。

3.1.5 数据修改 (Delete / Insert / Update)

1. Delete:

```
1 delete from instructor
2 where salary < (select avg(salary) from instructor);
```

注意：

- delete 在执行时，先将条件算完（应用删除前的数据），才会正式开始删除数据。
- 介词为 **from + 表名**；

2. Update:

```
1 UPDATE Student
2 SET Age = Age + 1
3 WHERE DeptID = 3
```

- 直接加上表名，不需要介词；
- Set 表示更新操作；

可以进一步使用 update + case 表示多个条件的更新：

```
1 update instructor
2 set salary = case
3     when salary <= 100000 then salary * 1.05
4     else salary * 1.03
5 end;
```

- 在 set 中使用 case，后续跟不同的条件下的更新表达式；

3. Insert:

```
1 INSERT INTO Student(StudentID, StudentName, DeptID, Age) VALUES (1001,
2 'Alice', 3, 19);
```

- Into 后跟表（也可以带上属性），Values 后跟具体值；

也可以插入一个集合（利用子查询）：

```
1 INSERT INTO HonorStudent (StudentID, DeptID)
2 SELECT StudentID, DeptID
3 FROM Student
4 WHERE Age > 20;
```

3.1.6 连接表达式 (Join)

类型	说明
inner join	只保留匹配
left outer join	左表全保留
right outer join	右表全保留
full outer join	全保留

```

1 SELECT S.StudentName, D.DeptName
2 FROM Student S
3 INNER JOIN Department D
4     ON S.DeptID = D.DeptID
5 WHERE D.DeptName = 'Computer Science';

```

- 其中，Join 条件可以用以下表示：
 - `natural join`：自动选择同名属性进行连接，使用时需要注意防止同名但语义不同的属性被错误等值；
 - `join ... on`：使用自定义条件；
 - `join ... using (A1, A2)`：指定同名属性进行连接；

3.2 DDL 数据定义语言

DDL: A special language used to specify a *database schema* by a set of definitions, also including *data storage and definition language*, which provides commands for *defining relation schemas, deleting relations, and modifying relation schemas*.

3.2.1 表定义 (Create / Drop / Alter)

1. create table

```

1 create table instructor (
2     ID char(5),
3     name varchar(20) not null,
4     dept_name varchar(20),
5     salary numeric(8,2),
6     primary key (ID),
7     foreign key (dept_name) references department
8 );

```

- primary key: 唯一 + 非空
- unique: 允许 null
- foreign key: 默认 **restrict** (拒绝删除或修改父表行)，注意还有以下外键约束：

- **on delete/update cascade**: 当外键所在的表的记录被删除/更新时, 该记录也被级联删除/更新;
- **on delete/update set null/default**: 当外键所在的表的记录被删除时, 该记录的外键字段被设为 null 或默认值;
- check (P): 单表约束

2. alter table

```
1 alter table r add A D
```

- A - 属性名称, D - 属性类型;
- 只能 **加属性**, 旧元组该属性值为 null

3. drop table

```
1 drop table r
```

3.3 DCL 数据控制语言

主要是一些权限/安全性控制。

3.3.1 Authorization 授权

```
1 grant select on instructor to U1, U2;  
2 revoke select on instructor from U1 cascade;
```

4 其他数据库对象及其 SQL

4.1 视图 View

4.1.1 基本概念

视图是从一个或几个基本表导出的表。它本身不独立存储在数据库中, 即**数据库中只存放视图的定义而不存放视图对应的数据**, 这些数据仍存放在导出视图的基本表中, 因此视图是一个**虚表**。视图在概念上与基本表等同, 用户可以在视图上再定义视图。

4.1.2 视图 SQL

1. Create View:

```
1 create view faculty as
2 select ID, name, dept_name
3 from instructor;
```

- 介词为 **as**，后面跟一个查询（选出要放在视图中的数据）；

2. Update View:

```
1 insert into faculty values ('30765', 'Green', 'Music');
```

对视图的更新时通过作用到原关系集合来实现的，其相当于在原关系集合中执行上诉操作。

3. Delete View:

```
1 DROP VIEW 视图名称;
```

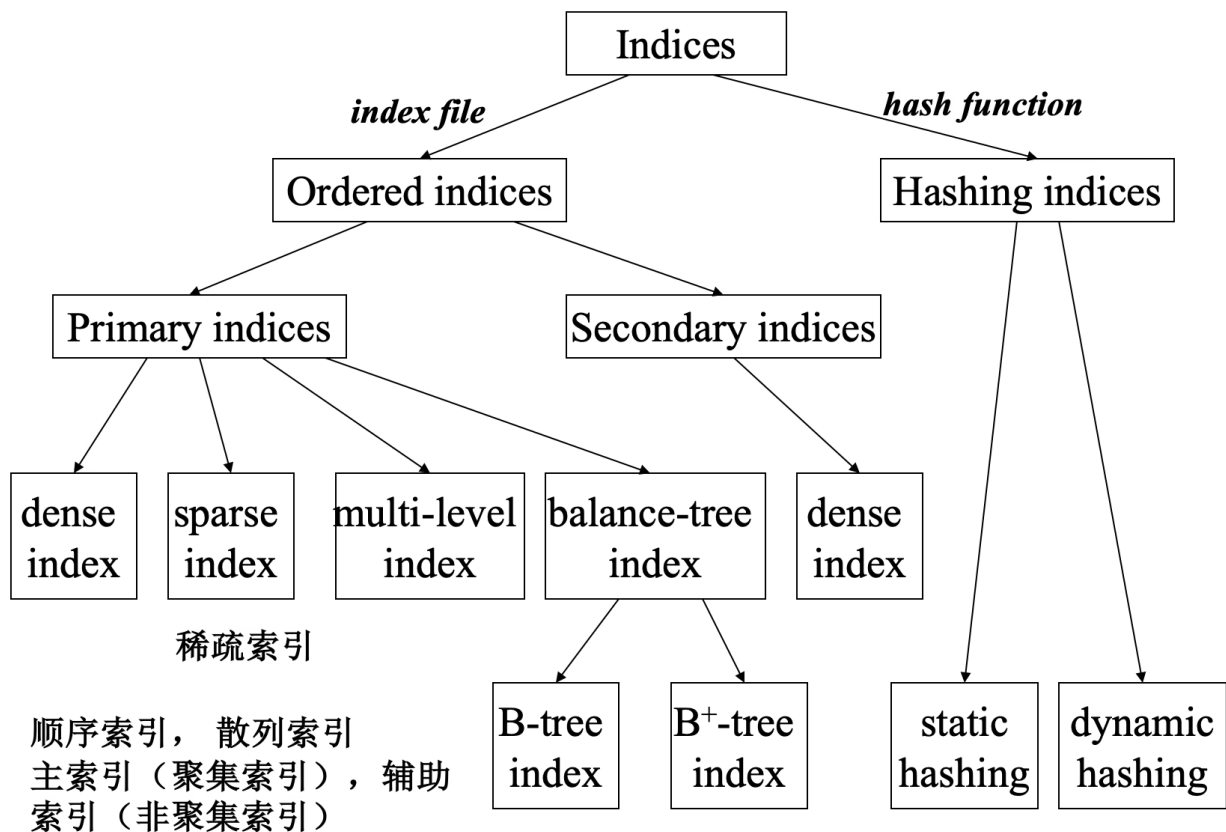
4.2 索引 Indexing

4.2.1 基本概念

索引文件由若干 **index entry(索引项)** 组成，形式为： `(search-key, pointer)`，其中：

- 搜索键（search-key）：用于查找记录的一个或多个属性；
- pointer 指向：数据记录或包含该记录的数据块；

4.2.2 分类



类型	特点	适用范围
Ordered Indices (顺序索引)	search key 有序存储	支持等值、范围查询
Hash Indices (散列索引)	通过 hash function 分布到桶	只支持等值查询

类型	特点	适用范围
Dense Index (稠密索引)	每个 search-key 值都有索引项	辅助索引
Sparse Index (稀疏索引)	数据文件必须按 search key 顺序存储	通常用于主索引

稀疏索引的查找流程如下：

1. 在索引中找到 最大且 $< K$ 的 search-key
2. 从该指针指向的位置 顺序扫描数据文件

4.2.3 Ordered Indices 顺序索引

有序索引是指索引文件中的记录按照**搜索键**的大小顺序排列的索引。换句话说，索引表本身是有序的，可以利用二分查找或顺序访问快速定位数据。进一步可以分为：

Primary Index/Clustering Index (主索引/聚集索引)：

- search key 的顺序 = 数据文件的物理顺序，即 Search Key 建立在有序域上；
- 一个文件 **最多只有一个** primary (clustering) index；
- search key **通常是主键，但不是必须**；
- **Index-sequential file** = 顺序文件 + 主索引；

Secondary Index（辅助索引 / 非聚集索引）：

- search key 顺序 $\neq$ 数据文件顺序，即 Search Key 建立在无序域上；
- 可以有多个辅助索引；
- **一定是稠密索引**；

4.2.4 B⁺ -Tree Index

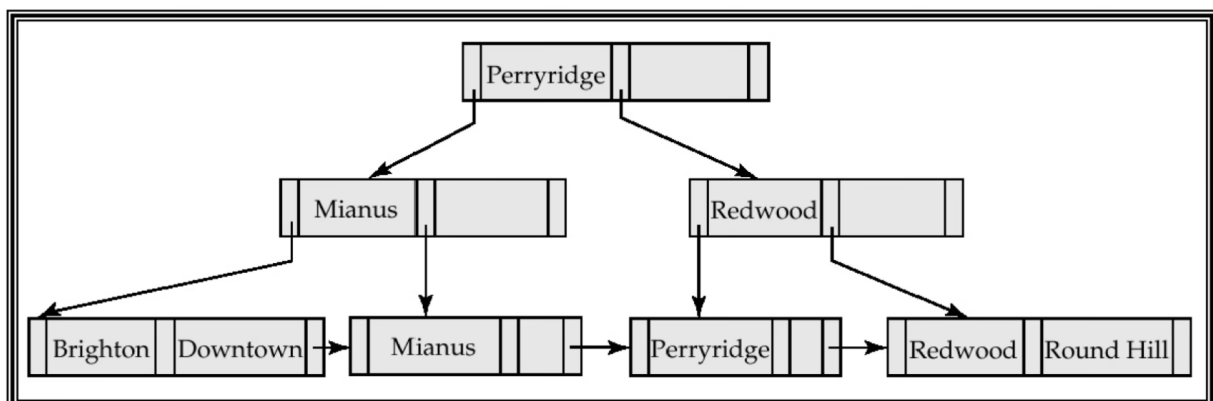
- 所有 **根 $\rightarrow$ 叶** 路径等长
- 非叶节点：
 - children $\in [\lceil n/2 \rceil, n]$
- 叶节点：
 - values $\in [\lceil (n-1)/2 \rceil, n-1]$
- 叶节点 **按 search-key 顺序链表连接**，即 $K_1 < K_2 < \dots < K_{n-1}$
- **叶节点稠密，非叶节点稀疏**

节点结构形如：

P1		K1		P2		K2		...		Kn-1		Pn
----	--	----	--	----	--	----	--	-----	--	------	--	----

，其中：

- Ki: search key
- Pi:
 - 非叶：指向子节点
 - 叶节点：指向记录或记录桶



4.2.5 Hash Indices 散列索引

了解即可。

静态哈希的一些缺点：

- 数据增长 → 哈希溢出（碰撞）严重
- 数据减少 → 空间浪费
- 重建 hash 代价极高

实际 DBMS 中很少支持哈希索引。

4.2.6 Multiple-Key 复合索引

左前缀原则：假如在 `(a, b, c)` 这些字段上建立了复合索引，查询加速有效的情况为

1	a
2	a, b
3	a, b, c

即必须从左边的属性开始，不能跳过中间的列，才能充分利用索引。

同时，如果从某一列开始使用范围查询，则右边的属性索引都无法生效。

例如：`WHERE a > 1 AND b = 2` 会导致列 b 索引失效。

4.2.7 索引 SQL

1	<code>create [unique] index index_name on table_name(attribute_list);</code>
2	<code>drop index index_name;</code>

5 数据库设计

5.1 基本流程

1. 需求分析 (User Requirements analysis);
2. 概念设计 (Conceptual Schema design): The entity-relationship model (*E-R*) is typically used to represent the conceptual design.
 - 使用 E-R 模型表示概念设计;

3. **逻辑设计 (Logical design)**: The implementation data model is typically the relational data model, and this step typically consists of mapping the conceptual schema defined using *the entity-relationship model into a relation schema*.
- 将 E-R 模型转换为关系模式;
 - 视图设计;
4. **物理设计 (Physical design)**: 文件结构, 索引等;

5.2 基本概念

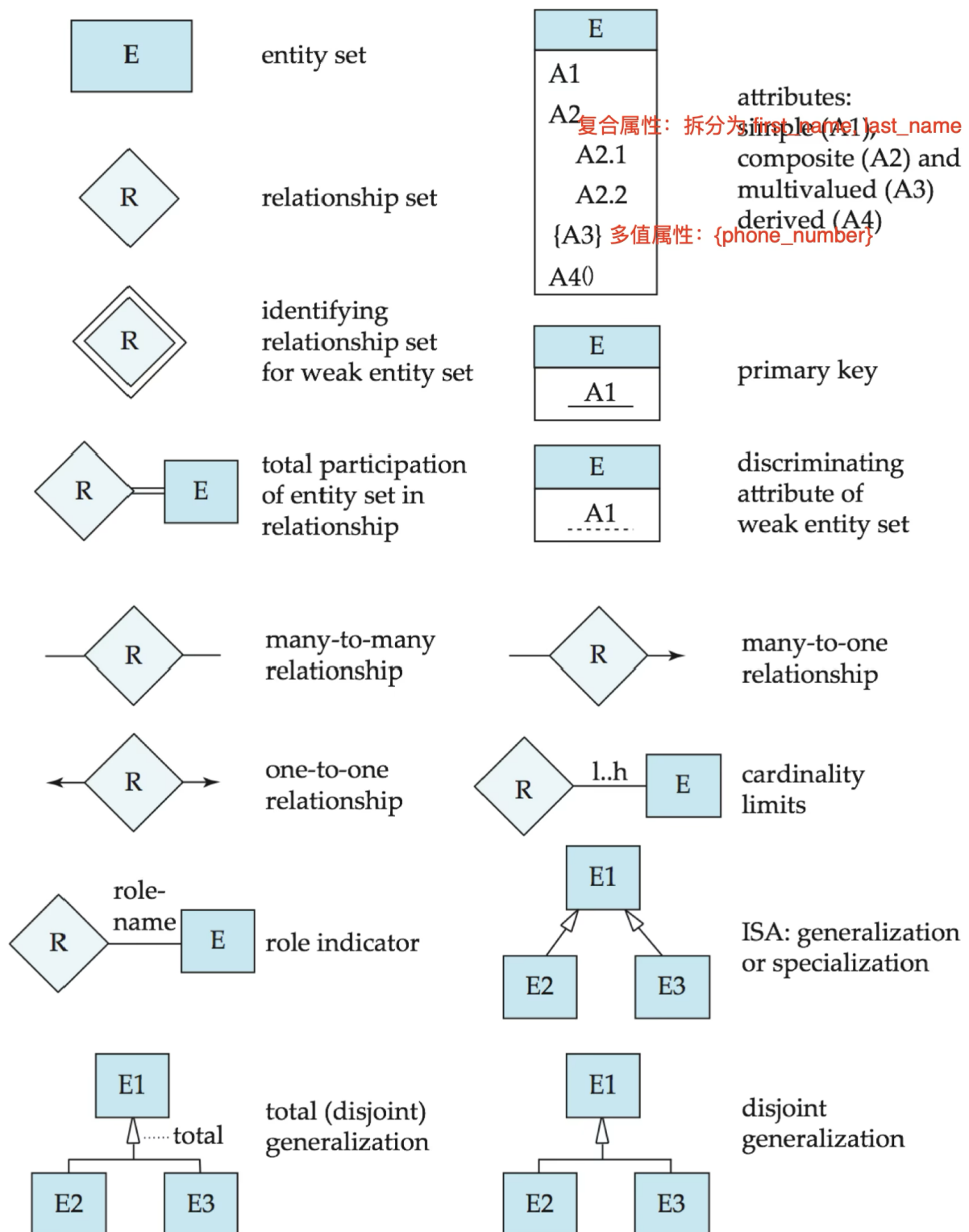
- E -> Entity Set(实体集): 同类实体的集合;
 - **弱实体集 (Weak Entity Sets)** 指没有足够属性能唯一标识自身的实体集合, 必须依赖另一个实体集 (**强实体集**) 来唯一标识。
 - 弱实体可能有自己的标识符, 但只能在依赖强实体的上下文中唯一, 也称“**部分键 (discriminator)**”。
- R -> Relationship Set(联系集): 是同类关系的集合。每个关系集包含若干关系 (relationship), 每个关系把若干实体联系起来。
 - **度**: 参与每个关系的实体集数量, e.g. 二元关系;
- Attributes(属性);

5.3 映射基数 Mapping Cardinality Constraints

Mapping Cardinality Constraints: 描述一个实体在某个关系集中能参与该关系的数量, 即一个实体最多能与多少个另一个实体集的实体相关联。

类型	描述	示例
一对一 (1:1)	每个实体仅与另一个实体对应	每个部门有且仅有一个经理，每个经理只管理一个部门
一对多 (1:N)	实体集 A 的一个实体可以关联实体集 B 的多个实体，但 B 的每个实体只能关联 A 的一个实体	一个教授可以教多门课程，但每门课程只有一个教授
多对一 (N:1)	与一对多相反	多门课程由同一个教授授课，每个教授教多门课
多对多 (M:N)	实体集 A 的实体可以关联 B 的多个实体，B 的实体也可以关联 A 的多个实体	学生选修课程：一个学生选多门课，一门课程被多个学生选

5.4 E-R 图绘制



5.5 最小 - 最大基数表示法 (l.h)

这是 E-R 模型中一个易错点, l.h 表示, 该侧的实体集最小/最多能参与联系集的数量, 以 [A - 3..5 - Relationship Set - 0..1 - B](#) 为例:

- 3..5 说明，A 中的每个实体，至少在 Relationship 中出现了 3 次；
- 0..1 说明，B 中的每个实体，至多在 Relationship 中出现了 1 次；

因此，(A1, B1), (A1, B2) 是允许的，即一个 A 能对应多个 B，但一个 B 只能对应一个 A，因此这是一对多的关系，与表示中的直觉相反。

5.6 扩展 E-R 特性

了解即可：

- **Specialization(特化)**：自顶向下
- **Generalization(泛化)**：自底向上

根据这些低层实体集是否相交，即是否存在公共实体，进一步划分为：

- **Disjoint (正交/互斥)**：一个实体最多只能属于一个子类。
 - e.g. 一个教师不可能既是教授又是副教授。
- **Overlapping (重叠)**：一个实体可以属于多个子类。

根据超类的所有实体是否都必须属于某个子类，划分为：

- **Total specialization (完全演绎/特化)**：超类中的每个实体都必须出现在某个子类中。
 - 例如，一个教师只可能既是教授、副教授、讲师、助教之一。
- **Partial specialization (部分演绎/特化)**：超类中有些实体可能不属于任何子类。

5.7 E-R 模型转换为关系模式

- **实体集 → 关系**：
 - 强实体：直接变表；
 - 弱实体：加上识别实体主键，部分键和依赖实体主键组合成为表的主键；
 - 复合属性 → 把它们的组成部分单独展开为独立列；
 - 多值属性：单独建表，形如 (实体主键, 多值属性)，主键为二者组合；
- **联系集**：
 - **M:N** → 关系独立建表，用两侧实体集的主键组合作为主键 + 联系集的属性；

- **1:N 且 N 端是全参与** → 在 N 端加外键，指向 1 端的主键，同时在 N 端加关系集属性；
- **1:1** → 可以选择**全参与的一端**一边加外键和联系集属性；

注意如果 1:N 或 1:1，选择的 N 端不是全参与，也需要像 M:N 一样单独建表。

5.8 非二元关系转换为二元关系

假设有一个三元关系 $R(A, B, C)$ ：

1. 把关系 R “提升”为一个新的实体集 E （类似 **Aggregation**）， E 代表了原来 R 中的每一条关系记录。
2. 创建三个新的二元关系： $RA(E, A)$ ， $RB(E, B)$ ， $RC(E, C)$ ，这三个关系分别连接 E 与原来的实体 A 、 B 、 C 。
3. 给 E 一个唯一属性（主键），比如 E_id ，然后把原本 R 的属性（比如日期、评分等）都作为 E 的属性。
4. 假设原来在 R 中有一个三元关系记录： (a_i, b_i, c_i) ：
 - (a) 在 E 中新建一个实体 e_i ；
 - (b) 在 RA 中添加 (e_i, a_i) ；
 - (c) 在 RB 中添加 (e_i, b_i) ；
 - (d) 在 RC 中添加 (e_i, c_i) 。

6 关系规范化

- 在逻辑设计阶段对得到的关系模式进行范式识别和分解。
- 主要是解决
 - 插入异常；
 - 删除异常；
 - 数据冗余；

6.1 函数依赖 (Functional Dependencies, FD)

定义：在关系模式 R 中，若任意两个元组在属性集 α 上取值相同，则在属性集 β 上取值也相同，记为： $\alpha \rightarrow \beta$ ，读作 α 函数确定 β ，或 β 函数依赖于 α ，形式化描述为：

1	$t1[\alpha] = t2[\alpha] \Rightarrow t1[\beta] = t2[\beta]$
---	---

显然：

- **超键 (Superkey)** : $K \rightarrow R$
- **候选键 (Candidate Key)** : $K \rightarrow R$, 且任意真子集都不能决定 R

类型：

1. **平凡依赖 (Trivial FD)** : 若 $\beta \subseteq \alpha$, 则 $\alpha \rightarrow \beta$ 是平凡的, e.g. $(ID, name) \rightarrow ID$, 显而易见, 所以平凡;
2. **传递依赖 (Transitive Dependency)** : 若 $\alpha \rightarrow \beta$, $\beta \rightarrow \gamma$, 且 β 不是候选键 (β 不是候选键 $\iff \beta \not\subseteq \alpha \wedge \beta \nrightarrow \alpha$), 即 β 既不是 α 的子集, 也不能反过来决定 α), 则 γ 传递依赖于 α ,
3. **部分依赖 (Partial Dependency)** : 非主属性只依赖于候选键的一部分;

6.2 三个公理

- **自反律 (Reflexivity)** : 若 $\beta \subseteq \alpha$, 则 $\alpha \rightarrow \beta$
- **增广律 (Augmentation)** : 若 $\alpha \rightarrow \beta$, 则 $\gamma\alpha \rightarrow \gamma\beta$
- **传递律 (Transitivity)** : 若 $\alpha \rightarrow \beta$ 且 $\beta \rightarrow \gamma$, 则 $\alpha \rightarrow \gamma$

补充：

- **union (合并)** : $\alpha \rightarrow \beta, \alpha \rightarrow \gamma \Rightarrow \alpha \rightarrow \beta\gamma$;
- **Decomposition (分解)** : $\alpha \rightarrow \beta\gamma \Rightarrow \alpha \rightarrow \beta, \alpha \rightarrow \gamma$
- **Pseudotransitivity (伪传递性)** : $\alpha \rightarrow \beta, \gamma\beta \rightarrow \delta \Rightarrow \alpha\gamma \rightarrow \delta$;

6.3 如何计算函数依赖集闭包

F^+ 表示 F 的闭包, 表示由 F 可以推导出的所有函数依赖的集合。

1. 对 F^+ 中每个依赖应用自反性与增广规则 $\rightarrow$ 扩大 F^+
 - 例如从 $A \rightarrow B$: 反射产生 $A \rightarrow A$, 增广产生 $AC \rightarrow BC, AD \rightarrow BD...$
2. 对 F^+ 中任意两个依赖应用传递规则 $\rightarrow$ 扩大 F^+
 - 例如: 如果 $A \rightarrow B$ 、 $B \rightarrow C$ 都在 F^+ , 则加入 $A \rightarrow C$
3. 重复, 直到 F^+ 不再增加

6.4 如何计算属性集闭包

定义：给定一个属性集 X 和一组函数依赖 F ， X 的属性集闭包记为 X^+ ，它表示：在依赖集 F 下，由 X 能决定的所有属性的集合。

计算方法：

1. 初始化： $X^+ = X$;
2. 对每个函数依赖 $A \rightarrow B \in F$:
 - 若 $A \subseteq X^+$ ，则把 B 加入 X^+ ，即 $X^+ = X^+ \cup B$;
3. 重复直到 X^+ 不再变化;

用途：

- 判断 **是否为超键**：如果 X 能唯一确定关系模式 R 的全部属性，即 X^+ 包含 R 的全部属性，则 X 是超键；
- 判断 $\alpha \rightarrow \beta$ **是否成立**：判断 $\alpha \rightarrow \beta$ 是否被 F 逻辑蕴含，可以通过 $\beta \subseteq \alpha^+$ 来判断；
- 构造 **候选键**
- 推导 F^+ ：对每个 $\gamma \in R$ ，计算其闭包 γ^+ ，对于属性 $S \in \gamma^+$ ，可以得到函数依赖 $\gamma \rightarrow S$ ；

6.5 如何求最小覆盖 / 正则覆盖 (Canonical Cover)

无关属性：

- 左部无关：对于属性 A ，如果 $A \in \alpha$ ，并且 $(F - \alpha \rightarrow \beta) \cup \{(\alpha - A) \rightarrow \beta\}$ 逻辑蕴含 F ，即删掉它后，函数依赖 $\alpha \rightarrow \beta$ 依旧成立，则 A 为无关/冗余属性；
- 右部无关：对于属性 A ，如果 $A \in \beta$ ，并且 $(F - \alpha \rightarrow \beta) \cup \{\alpha \rightarrow (\beta - A)\}$ 逻辑蕴含 F ，即删掉它后，函数依赖 $\alpha \rightarrow \beta$ 依旧成立，则 A 为无关/冗余属性；

判断属性是否为无关属性：

对于函数依赖 $\alpha \rightarrow \beta \in F$ ：

1. 判断 $A \in \alpha$ 是否冗余：
 - (a) 使用 F ，计算 $(\alpha - A)^+$ ；
 - (b) 如果 $(\alpha - A)^+(F)$ 包括 β ，则 A 冗余；
2. 判断 $A \in \beta$ 是否冗余：

- (a) 使用 $(F - \{\alpha - \beta\}) \cup \{\alpha \rightarrow (\beta - A)\}$, 计算 α^+ ;
- (b) 如果 $\alpha^+(F')$ 包含 A , 则 A 冗余;

计算正则覆盖:

1. 初始化 $F_c = F$
2. 合并右边: 如果有 2 条依赖 $\alpha_1 \rightarrow \beta_1, \alpha_1 \rightarrow \beta_2$, 则合并为 $\alpha_1 \rightarrow \beta_1\beta_2$;
3. 寻找所有函数依赖左部和右部的无关属性;
4. 去除所有无关属性

6.6 范式

1NF: 所有属性是原子的, 不可再分的。

2NF: 消除非键属性对候选键的部分依赖。即, 在 1NF 的基础上, 满足每个非键属性完全依赖于候选键。

3NF: 消除非键属性对候选键的传递依赖。即, 在 2NF 的基础上, 满足每个非键属性不传递依赖于候选键。

BCNF: 消除键属性对候选键的部分和传递依赖。即, 在 3NF 的基础上, 键属性也满足直接并且完全依赖于每个不包含它自身的候选键。

6.7 如何计算候选键

1. 将 R 的属性分为以下 4 类, 并令 X 代表 L 和 N 类, Y 代表 LR 类;
 - (a) L 类: 仅出现在 F 的函数依赖左部的属性;
 - (b) R 类: 仅出现在 F 的函数依赖右部的属性;
 - (c) N 类: 在 F 的函数依赖左右两边均未出现的属性;
 - (d) LR 类: 在 F 的函数依赖左右两边均出现的属性;
2. 计算 X^+ , 若包含了所有关系模式 R 的属性, 则 X 为 R 的唯一候选关键字, 停止, 得到结果, 转 5, 否则转 3;
3. 在 Y 中取一属性 A , 求 $(XA)^+$, 若它包含了 R 的所有属性, 则转 4, 否则, 调换一属性反复进行这一过程, 直到试完所有 Y 中的属性;
4. 如果已找出所有的候选关键字, 则转 5, 否则在 Y 中依此取两个、三个, ..., 求他们的属性闭包, 直到其闭包包含 R 的所有的属性。

6.8 如何判断无损连接分解 Lossless-join decomposition

假如 R 被分解为 R_1, R_2 ，该分解是无损连接当且仅当满足以下条件之一：

- $R_1 \cap R_2 \rightarrow R_1$
- $R_1 \cap R_2 \rightarrow R_2$

即交集要么是 R_1 的超键，要么是 R_2 的超键。

6.9 如何判断是否函数依赖保持 Dependency preservation

判断方式1（使用定义）： 假如关系模式 R ，函数依赖 F ，分解为 $R_1, R_2, \dots, R_n$ ，对应的函数依赖为 $F_i = \{X \rightarrow Y \mid X \rightarrow Y \in F^+ \cap XY \subset R_i\}$ ，如果满足以下条件，则函数依赖保持：

$$(F_1 \cup F_2 \cup \dots \cup F_n)^+ = F^+ \quad (1)$$

判断方式2：

1. 对每个函数依赖 $\alpha \rightarrow \beta \in F$ ，初始化 result 为所有左部的 α ；
2. 对分解后的每一个模式 R_i ，将 result 与 R_i 取交集后的闭包，再和 R_i 取交集，将该结果加入 result
3. 重复直到 result 不再变化，判断结果是否包含 β ，如果包含，则保持函数依赖；

6.10 如何分解为 3NF

1. 求 F 的最小覆盖 F_c
2. 对 F_c 中每个 $\alpha \rightarrow \beta$ 建一个关系 $\alpha \cup \beta$
3. 若没有关系包含候选键 $\rightarrow$ 加入一个候选键关系

保证：

- 每个子模式是 3NF
- 无损连接
- 依赖保持

6.11 如何分解为 BCNF

1. 反复找 违反 BCNF 的 FD

2. 分裂为:

- $(\alpha \cup \beta)$
- $(R - (\beta - \alpha))$

BCNF 分解可能丢失依赖保持。

7 物理存储结构

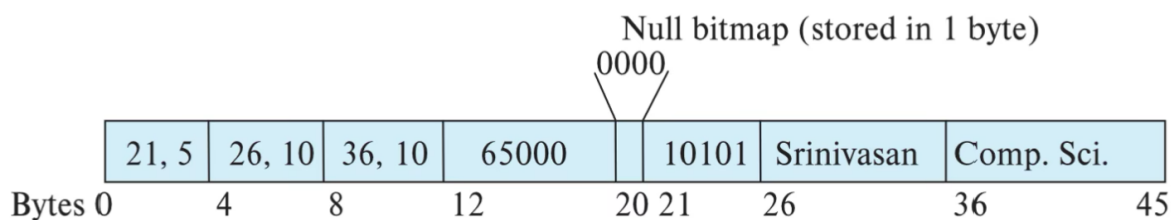
7.1 File organization 文件组织

Fixed-Length Records (定长记录)

- 第 i 条记录起始地址: $n \times (i - 1)$;
- 其中, n 为单条记录长度;
- 删除记录的三种方式:
 1. 整体前移 (压缩);
 2. 用最后一条记录填补;
 3. **Free List (空闲记录链)** :
 - 文件头存: 第一个被删除记录地址
 - 被删除记录中: 存“下一个空闲记录”的地址
 - 不额外占用空间

Variable-Length Records (变长记录)

- 先放固定长度部分
- 变长属性用 (offset, length)
- 实际变长数据存放在后部
- **Null-value bitmap** 表示空值



7.2 Organization of Records in Files 文件中的记录组织

Heap File（堆文件）：无主键、且无索引的关系表组织为 heap 文件；

- 记录可放在任意有空闲空间的位置
- 通常不移动记录
- 使用 **Free-space map**
 - 每个 block 一个 entry
 - 可有二级 free-space map

Sequential File（顺序文件）：有索引的关系表组织为顺序文件；

- 按 **搜索键排序**
- 适合全表顺序扫描
- **插入**：
 - 有空位 → 插入
 - 无空位 → overflow block + 指针链
- **删除**：
 - 指针链
- 需要 **周期性重组（reorganization）**

Hashing File（哈希文件）：根据 hash(key) 定位 block，适合等值查询；

Clustering File（聚集文件）：多个关系（即多种记录）存储在同一文件；

- 相关记录物理相邻 → 减少 I/O；

7.3 Data Dictionary Storage 数据字典存储

Data Dictionary（也叫 **system catalog**，系统目录），用于存储 metadata，包括：

- 表名、属性名、类型
- 视图定义
- 约束
- 用户信息
- 统计信息
- 文件物理组织方式
- 索引信息

7.4 Column-Oriented Storage 列存储

- 将每一列（单个属性）单独存储
- 适合分析型查询（OLAP）
- 能更好压缩存储，但 tuple 重构代价高

区别于行存储，一般的行存储适合于 OLTP（查询、插入、更新、删除等）基本业务逻辑）。

8 查询处理和优化

8.1 基本流程

1. Parsing and Translation（语法分析与翻译）：

- (a) 将 SQL 查询 翻译成 内部表示；
- (b) 再翻译成 关系代数表达式；

2. Optimization（查询优化）：

- (a) 逻辑优化（查询重写，启发式代数优化）；
- (b) 物理优化；

3. Evaluation（执行）；

8.2 查询成本估算

查询代价 $\approx$ 磁盘访问代价

- t_T : 传输一个磁盘块的时间
- t_S : 一次磁盘寻道时间
- b : 块传输次数
- S : 寻道次数

$$\text{Cost} = b \cdot t_T + S \cdot t_S \quad (2)$$

对于选择操作：

1. 线性查找 linear search：把整个表全部块都扫描一次（默认全部块连在一起，只寻道一次）

- 任意选择条件
- 是否有索引无关
- 文件是否有序无关
- 一般情况下: $b_r \cdot t_T + t_S$;
- 若是 **key 等值查询且为候选键** (即结果唯一, 且找到即可停止): $\frac{b_r}{2} \cdot t_T + t_S$ (期望)

2. 二分查找 binary search: 要求

- 等值查询
- 文件按该属性有序排序
- 块连续存储

对磁盘块进行二分查找, 每一次都要先寻道, 移动到目标块所在的轨道, 然后把该块的数据读上来, 以便比较 search-key。因此, 二分查找进行了 $\lceil \log_2 b_r \rceil$ 次比较, 每次比较都需要一次寻道 + 一次传输:

$$\lceil \log_2 b_r \rceil \cdot (t_T + t_S) \quad (3)$$

若有多条记录, 还需加上包含这些记录的块传输代价。

3. 索引查找 index search:

- **主索引 + 候选键 + 等值**: 等值且定义在候选键上的主索引, 说明结果只有一条记录, 因此先根据索引树找到对应索引, 然后读取数据块, 设 h_i 为索引树深度:

$$(h_i + 1) \cdot (t_T + t_S) \quad (4)$$

- **主索引 + 非键 + 等值**: 可能有多条记录, 用 b 表示结果块数:

$$h_i \cdot (t_T + t_S) + t_S + b \cdot t_T \quad (5)$$

- **辅助索引 + 等值**:

- 若是候选键 $\rightarrow$ 单条记录, 同上;
- 若是非候选键, 每条记录可能在不同块, 即索引查到的 n 个匹配记录可能都分别在不同块中, 寻道时间增加:

$$(h_i + n) \cdot (t_T + t_S) \quad (6)$$

4. 比较选择 ($<$, $\geq$) :

- 若是主索引，顺序扫描数据文件；
- 若是辅助索引：
 - 顺序扫描索引叶子
 - 每条记录一次 I/O

8.3 启发式代数优化

1. 从 SQL 构建查询树：

- FROM $\rightarrow$ 笛卡尔积 $\times$
- WHERE $\rightarrow$ 选择 σ
- SELECT $\rightarrow$ 投影 π

查询树形如：

- 叶节点：关系
- 非叶节点： σ 、 π 、 $\bowtie$ 、 $\times$ 等关系代数操作
- 实际的 SQL 执行顺序为自底向上执行

2. 利用选择串接律，拆分选择条件：

选择串接律：

$$\delta_{C_1 \text{ and } C_2 \dots C_n}(E) = \delta_{C_1}(\delta_{C_2} \dots (\delta_{C_n}(E)) \dots) \quad (7)$$

3. 选择下推：使其尽可能移到靠近叶节点，为了让选择操作尽量早执行；

主要利用以下规则：

- 选择交换律：（遇到选择）

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E)) \quad (8)$$

- 遇到连接：先选出必要的条件，再连接

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2)) \quad (9)$$

- 遇到集合操作：先选择必要的条件，在进行集合运算

$$\sigma_{\theta}(E_1 - E_2) = \sigma_{\theta}(E_1) - \sigma_{\theta}(E_2) \quad (10)$$

- 投影选择交换律：（遇到投影）如果选择的条件在投影的结果中

$$\Pi_L(\delta_\theta(E)) = \delta_\theta(\Pi_L(E)) \quad (11)$$

4. 投影下推 / 删除冗余投影：

- 投影串接律：仅保留一个最小投影即可

$$\pi_{L_1}(\pi_{L_2} \cdots (\pi_{L_n}(E)) \cdots) = \pi_{L_1}(E) \quad (12)$$

- 遇到连接：先投影出必要属性，再连接

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2)) \quad (13)$$

- 遇到并操作：

$$\Pi_L(E_1 \cup E_2) = (\Pi_L(E_1)) \cup (\Pi_L(E_2)) \quad (14)$$

- 投影选择交换律：

5. 使用选择、投影的串接律，选择、投影的交换律，将串接的多个选择和投影组合成一个

6. 利用笛卡尔乘积/连接/结合操作的结合律，按照小关系先执行的原则，安排操作顺序

7. 组合笛卡尔乘积和相继的选择操作作为连接操作： $\times + \sigma \rightarrow \bowtie$

8. 对每个叶节点加必要的投影操作，消除对查询无用的属性

9. 分组：识别可流水线执行的子树；

子树划分规则如下：每个二元运算（笛卡尔乘积、连接、集合操作）与其直接祖先（不越过别的二元运算节点）的一元运算节点（投影、选择），分为一组，如其子孙节点一直到叶节点皆为选择或投影，也并入该组。

9 事务

9.1 ACID 特性

- **Atomicity(原子性)**：要么全做，要么全不做，如果事务中途失败，部分更新不能反映到数据库中；

- **Consistency(一致性)**: 事务在“隔离执行”下必须保持数据库一致性;
- **Isolation(隔离性)**: 并发事务之间互不感知, 一个事务的中间结果不能被其他事务看到;
- **Durability(持久性)**: 事务一旦 commit, 即使系统崩溃, 修改结果也必须保留;

并发操作主要破坏了事务的隔离性。数据的持久性由恢复系统实现。

9.2 事务状态及其转换

1. **Active**: 正在执行
2. **Partially committed**: 最后一条语句执行完
3. **Failed**: 发现无法继续执行
4. **Aborted**: 回滚完成
 - 可选择: 重启 / 终止
5. **Committed**: 成功完成

9.3 Serializability 冲突可串行化

- 假设每个事务单独执行都符合一致性, 那么串行调度每个事务执行, 显然是安全的。
- 并发调度 只要等价于某个串行调度 $\Rightarrow$ 也是正确的

判断是否冲突可串行化 (Conflict Serializability) 方法: 构造 Precedence graph (前驱图)

- 构图规则:
 - 顶点 V: 所有事务
 - 边 $T_i \rightarrow T_j$ 若有:
 1. T_i 的 `write(Q)` 在 T_j 的 `read(Q)` 之前
 2. T_i 的 `read(Q)` 在 T_j 的 `write(Q)` 之前
 3. T_i 的 `write(Q)` 在 T_j 的 `write(Q)` 之前
- 调度是冲突可串行化 $\Leftrightarrow$ 前驱图无环;
- 无环 $\rightarrow$ 拓扑排序 = 等价串行顺序:
 1. 寻找入度为 0 的节点;
 2. 删除该节点和连接的边, 重复第一步;

3. 如果过程中发现入度为 0 的节点不止一个，说明存在多个等价的串行调度。

9.4 Recoverability 可恢复调度

判断是否为可恢复调度 (recoverable schedule) 方法：

- 若 T_j 读了 T_i 写的数据： T_i 必须先 commit, T_j 才能 commit;
- 否则如果 T_i 回滚而 T_j 已提交 $\rightarrow T_j$ 就会读到脏数据;

9.5 Cascadeless 无级联回滚

比可恢复要求更严格。

- 若 T_j 读 T_i 的数据： T_i 必须在 T_j read 之前就 commit;
- 无级联回滚一定是可恢复的;

10 并发控制

10.1 并发可能带来的问题

1. 丢失对数据的修改;
2. 不可重复读：由于 `update` 造成的不一致数据，或由 `insert/delete` 造成的不一致的数据;
3. 读“脏”数据;

10.2 两阶段锁协议 (Two-Phase Locking, 2PL)

判断方法：

1. Phase 1: Growing Phase (增长阶段)
 -  可以获得锁
 -  不能释放锁
2. Phase 2: Shrinking Phase (收缩阶段)
 -  可以释放锁
 -  不能再获得锁

作用：两阶段锁协议可以确保冲突可串行化。

如何构造冲突可串行化序列?

1. 识别每个事务的**锁点(Lock Point)**: 一个事务获得最后一个锁的时刻;
2. 所有事务可以按 **lock point** 先后顺序串行化调度;

问题: 普通的 2PL 不能避免死锁和级联回滚回滚。

10.3 严格两阶段锁协议 (Strict 2PL)

判断方法: 在普通 2PL 的基础上, 要求 **X 锁** 一直持有到 **commit / abort** 之后才能释放。

作用: 可以避免级联回滚。

10.4 强两阶段锁协议 (Rigorous 2PL)

判断方法: 在严格 2PL 的基础上, 要求**所有锁 (S + X)** 都持有到 **commit / abort** 之后才能释放。

如何构造冲突可串行化序列? : 事务按提交顺序串行化。

作用: 可以避免级联回滚。

10.5 锁转换 (Lock Conversion)

了解即可, 在 2PL 的基础上增加了:

1. 第一阶段 (Growing)
 - 可以 $S \rightarrow X$ (upgrade)
2. 第二阶段 (Shrinking)
 - 可以 $X \rightarrow S$ (downgrade)

仍然可以保证冲突可串行化。

10.6 多粒度锁 (Multiple Granularity Locking)

意向锁 (Intention Lock) : 表明该节点的子节点的意向。

锁	含义
IS	子节点加 S 锁
IX	子节点加 X 锁
SIX	本节点 S + 子节点 X

作用：在判断时，不用检查所有子节点，也能判断是否能加锁。通过相容性矩阵进行判断

	IS	IX	S	S IX	X
IS	✓	✓	✓	✓	×
IX	✓	✓	×	×	×
S	✓	×	✓	×	×
S IX	✓	×	×	×	×
X	×	×	×	×	×

多粒度加锁规则：

- 必须从根向叶子加锁
 - 如果想要给子节点 Q 加上 S 或 IS，则父节点必须先加上 IS 或 IX；
 - 如果你想在 Q 上加写相关的锁（X / IX / SIX），则父节点必须先加上 IX 或 SIX；
- 解锁顺序：叶子 → 根

- 仍然要满足 2PL

10.7 死锁

死锁预防 (Prevention)：了解即可

1. 预声明所有锁
2. 按固定顺序加锁
3. 超时回滚 (**Timestamp Protocol**)
 - **wait-die** (等待死亡协议, 非抢占)
 - 如果老事务需要的锁被新事务持有, 老事务 **等**;
 - 如果新事务需要的锁被老事务持有, 新事务 **回滚**, 过一段时间再重启;
 - 即, 如果出现等待, 一定是**老等新**;
 - **wound-wait** (伤害等待协议, 抢占)
 - 如果老事务需要的锁被新事务持有, **回滚新事务**, 老事务相当于抢占了新事务的锁;
 - 新事务可以等老事务;
 - 即, 如果出现等待, 一定是**新等老**;

死锁检测：了解即可, 通过**等待图**检查, 有环 $\Leftrightarrow$ 死锁。

死锁恢复：了解即可, 选择代价最小的事务作为 victim, 同时注意避免同一事务反复回滚 (如设置保底机制) $\rightarrow$ 防止饥饿。

11 恢复系统

11.1 故障分类

1. **Transaction Failure (事务失败)**：包括
 - **逻辑错误**：事务自身逻辑错误, 无法继续;
 - **系统错误**：如死锁, 被 DBMS 强制中止;
2. **System Crash (系统崩溃)**：断电、OS crash、DBMS crash
 - **Fail-stop 假设**：崩溃后系统停止, **非易失存储 (磁盘) 内容不损坏**;
3. **Disk Failure (磁盘失败)**：磁头损坏等, 磁盘数据丢失, 如果可以通过 checksum 检测到, 则通过备份恢复;

主要在于处理系统崩溃情况，通过日志机制，日志（log）一定存放在 stable storage：

类型	是否掉电保存	示例
Volatile storage	✗	主存、cache
Non-volatile storage	✓	磁盘、SSD
Stable storage	“理想化”	RAID，多副本

11.2 数据访问模型

- read(X): 从 buffer block → 私有变量 xi
- write(X): 从 xi → buffer block
- output(B): buffer block → disk（可能延迟执行）

不一致产生原因：已经完成 `write(Y)`，但是缓冲区未及时 `output(B)` 就崩溃，因此记录会丢失，系统处于不一致状态，此时需要恢复。

11.3 Log-based Recovery

日志格式：

1	<Ti start>
2	<Ti, X, V1, V2>
3	<Ti commit>
4	<Ti abort>

使用原则：Write-Ahead Logging (WAL)

日志可以缓冲在内存中，但必须满足：

1. 日志按顺序写出
2. `<Ti commit>` 必须强制刷盘（log force）
3. 数据块写回前，相关日志必须已刷盘

即在数据库块写回磁盘之前，所有相关的日志记录必须先写入 stable storage。

事务回滚（非 crash 情况）：当事务 Ti 主动 rollback：

1. 从日志末尾向前扫描
 - 反向扫描，因为同一数据项可能被多次修改，必须恢复到最早之前的状态

2. 对 `<Ti, X, V1, V2>`:
 - 写回 V1
 - 写 补偿日志 (CLR) : `<Ti, X, V1>`
3. 遇到 `<Ti start>`:
 - 写 `<Ti abort>`

系统崩溃后的恢复规则:

对于每个事务 T_i , 如果日志中:

- 有 `<Ti start>`, 无 commit/abort, 则执行 undo 操作;
- 有 `<Ti start>` 且有 commit/abort, 则执行 redo 操作;

带并发事务的恢复: 假设遵循严格 2PL, 且未提交事务的修改对其他事务不可见;

1. Redo Phase:

- (a) 找到最近的 `<checkpoint L>`, 此时 $\text{undo-list} = L$;
- (b) 从该检查点开始, 顺序扫描:
 - 如果遇到 `<Ti, Xj, V1, V2>` 或 `<Ti, Xj, V2>`, 则进行 redo 操作, 将 X_j 写为 V2;
 - 如果遇到 `<Ti, start>`, 则将 T_i 加入 L;
 - 如果遇到 `<Ti, commit>` 或 `<Ti, abort>`, 则将 T_i 移出 L;

2. Undo Phase:

- (a) 从日志末尾向后扫描, 回滚 undo-list 中的事务;
- (b) 如果遇到 `<Ti, Xj, V1, V2>` 且 $T_i \in L$, 则执行 undo 操作, 将 X_j 写为 V1, 并写入一条日志 `<Ti, Xj, V1>`;
- (c) 如果遇到 `<Ti start>` 且 $T_i \in L$, 则将 T_i 移出 L, 并写一条日志 `<Ti abort>`;
- (d) 当 L 为空时停止;